

Reflective Programming

From Introspection to Proxies and Interception

Course block

Definition and motivation

Introspection and Interception

Proxies and AOP

Reflective Programming in Python

Pedro García López · TAP

Learning outcomes

By the end of this block, students should be able to explain and implement basic reflection patterns.

1 • Explain

Define reflection, reification, introspection, invocation and intercession.

2 • Use

Inspect classes and annotations, and invoke methods dynamically in Java.

3 • Design

Recognize when reflection is appropriate in frameworks, plugins and testing.

4 • Intercept

Build a tiny dynamic proxy for logging or authorization.

5 • Evaluate

Reason about costs: performance, type safety, security and maintainability.

Why reflection still matters

Most students have already used reflection indirectly, even if they have never called `java.lang.reflect`.

JUnit

find `@Test` methods

Spring

inject dependencies

Hibernate

map classes to tables

Jackson

serialize objects

Mockito

generate mocks

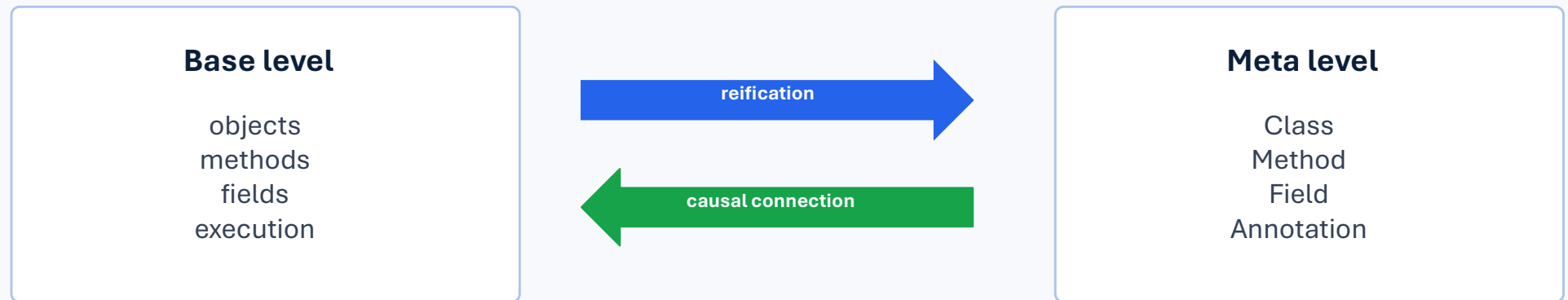
IDE/debugger

inspect runtime state

Teaching angle: start from frameworks students know, then reveal the mechanism behind them.

Core model: base level and meta level

Reflection creates a representation of the running program that the program itself can inspect or use.



Question for students: What changes when the representation is only readable vs modifiable?

Vocabulary: three capabilities

Separate the ideas clearly; many texts use “reflection” too broadly.

Introspection

Observe program structure or state.

Examples:

- list methods
- read annotations
- inspect object fields

Dynamic invocation

Call a method or construct an object through its meta-representation.

Examples:

- `Method.invoke(...)`
- `Constructor.newInstance(...)`

Intercession

Alter or intercept behavior.

Examples:

- dynamic proxies
- class loaders
- bytecode weaving
- AOP advice

Important distinction: Java reflection is strong for introspection, more limited for true intercession.

Reflection and metaprogramming

A useful map for organizing the rest of the topic.

Technique	When?	Examples
Runtime reflection	Inspect/use metadata at execution time	Java reflection, Python getattr
Annotations	Attach metadata to program elements	@Test, @Entity, @Autowired
Annotation processing	Generate/check code at compile time	Lombok, Dagger, MapStruct
Bytecode generation/weaving	Create or modify classes	Byte Buddy, CGLIB, AspectJ
Macros/templates	Generate code from code	Rust macros, C++ templates

Java reflection API: the mental model

Every program element has a runtime representation object.

Class<T>

Type metadata: name, superclass, interfaces, constructors.

Field

Metadata and access to object/class fields.

Method

Metadata and dynamic invocation.

Constructor

Dynamic object creation.

```
Class<?> c = Class.forName("demo.User");

for (Method m : c.getDeclaredMethods()) {
    System.out.println(m.getName());
}

User u = (User) c.getConstructor().newInstance();
```

Example 1: inspect a class

A simple live demo: discover structure without hard-coding it.

```
record User(Long id, String name) {  
    @JsonIgnore  
    public boolean isInternal() { return false; }  
}  
  
Class<?> clazz = User.class;  
System.out.println(clazz.getSimpleName());  
  
for (Method m : clazz.getDeclaredMethods()) {  
    System.out.println(m.getName());  
    for (Annotation a : m.getAnnotations())  
        System.out.println("  " + a.annotationType().getSimpleName());  
}
```

Discussion

How does a library like Jackson know what to serialize?

Concept

Metadata is data about the program itself.

Limitation

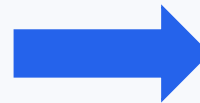
Reflection can bypass normal readability and type-checking assumptions.

Annotations: structured metadata

Annotations are the bridge from source code to frameworks.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Route {
    String path();
    String method() default "GET";
}

class UserController {
    @Route(path = "/users")
    public List<User> listUsers() { ... }
}
```



Source code

The programmer declares intent with @Route.

Runtime scan

Framework finds annotated methods.

Behavior

Framework registers a route automatically.

Mini-framework pattern: scan → select → invoke

This is the hidden algorithm behind many frameworks.



```
for (Method m : controller.getClass().getDeclaredMethods()) {  
    Route route = m.getAnnotation(Route.class);  
    if (route != null) {  
        router.register(route.path(), request ->  
            m.invoke(controller, request));  
    }  
}
```

Dynamic invocation: power and danger

The compiler can no longer check everything for you.

```
Method m = service.getClass().getMethod("findUser",  
Long.class);  
Object result = m.invoke(service, 42L);
```

Power

Late binding: decide which method to call using configuration, plugins or annotations.

Cost

Slower dispatch, more exceptions, less help from the compiler.

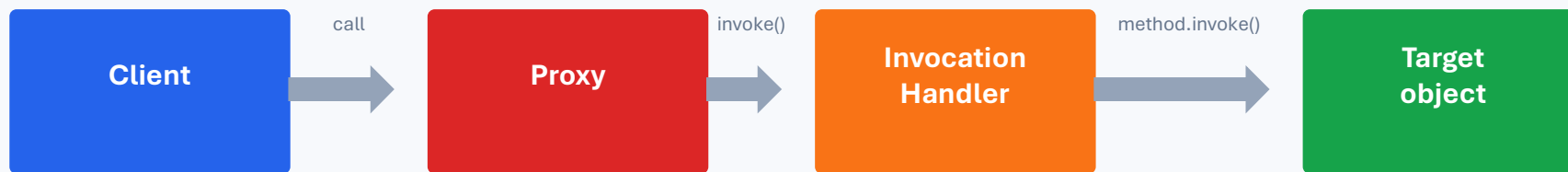
Risk

Can break encapsulation or expose private internals if used carelessly.

Rule of thumb: isolate reflection at framework boundaries; keep application code strongly typed.

Dynamic proxies: method interception

A proxy is an object with the same interface that routes calls through an `InvocationHandler`.



Use cases

Logging, authorization, transactions, retries, metrics.

Constraint

Standard Java dynamic proxies work through interfaces.

Frameworks

Spring AOP, RPC clients, mocks, decorators.

Example 2: logging proxy

A concise version students can implement in class.

```
class LogHandler implements InvocationHandler {
    private final Object target;

    LogHandler(Object target) { this.target = target; }

    public Object invoke(Object proxy, Method m, Object[] args)
        throws Throwable {
        long t0 = System.nanoTime();
        try {
            return m.invoke(target, args);
        } finally {
            long ms = (System.nanoTime() - t0) / 1_000_000;
            System.out.println(m.getName() + " took " + ms + " ms");
        }
    }
}
```

```
Service s = (Service) Proxy.newProxyInstance(
    Service.class.getClassLoader(),
    new Class<?>[] { Service.class },
    new LogHandler(new RealService())
);

s.findUser(42L);
```

Teaching activity: ask students to add authorization or retry logic without touching RealService.

From proxies to Aspect-Oriented Programming

AOP generalizes interception for cross-cutting concerns.

Cross-cutting concern

Behavior repeated across many modules: logging, tracing, security, transactions.

Pointcut

A rule selecting join points, for example “all methods annotated @Transactional”.

Advice

Code executed before, after or around selected join points.

Business code

security

transaction

logging

metrics

Key message: AOP improves modularity only if aspects remain explicit, documented and testable.

AOP example: timing selected methods

An aspect keeps cross-cutting code out of the business method.

Business code stays clean

```
@Timed
public Order findOrder(long id) {
    return repository.findById(id);
}
```

Aspect code is centralized

```
@Aspect
class MetricsAspect {
    @Around("@annotation(Timed)")
    Object measure(ProceedingJoinPoint pjp)
        throws Throwable {
        long t0 = System.nanoTime();
        try { return pjp.proceed(); }
        finally { record(pjp, System.nanoTime() - t0); }
    }
}
```

Runtime call path

Client

Spring/JDK
proxy

@Around
advice

Target
object

Pointcut

selects methods annotated @Timed

Advice

wraps the call before/after/around execution

Join point

the concrete method invocation being intercepted

Discussion question: when is this cleaner than an explicit decorator or middleware?

Costs and pitfalls

Reflection is powerful, but it is not free.

Performance

Reflective lookup and dynamic dispatch are usually slower than direct calls. Cache metadata when possible.

Type safety

Many errors move from compile time to runtime: wrong name, wrong parameters, wrong cast.

Encapsulation

setAccessible and deep reflection can break module boundaries and invariants.

Security

Scanning and invoking unknown code may expose attack surfaces in plugin systems.

Maintainability

Hidden control flow makes debugging harder: “who called this method?”

Compatibility

APIs such as native images, modules or ahead-of-time compilation may require explicit reflection configuration.

Design principle: make reflection a small, well-tested infrastructure layer — not a general coding style.

Decision guide: should I use reflection?

A pragmatic slide for software design discussions.

Good reasons

Framework infrastructure, dependency injection, testing tools, serializers, plugin loading, adapters to external schemas.

Bad reasons

Avoiding interfaces, hiding design problems, calling private methods because the model is inconvenient.

Prefer alternatives first

Interfaces, generics, sealed types, records, pattern matching, explicit registries.

When unavoidable

Centralize, cache metadata, validate early, document assumptions, test failure cases.

Exam question idea: compare an explicit plugin registry with a reflection-based plugin scanner.

Mini-lab: build a tiny test runner

Students implement a simplified JUnit to understand reflection by doing.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyTest { }

class CalculatorTest {
    @MyTest
    void additionWorks() {
        assertEquals(4, 2 + 2);
    }
}
```

Find methods annotated with @MyTest.
Create an instance of the test class.
Invoke each test method dynamically.
Catch exceptions and print PASS/FAIL.
Optional: add @BeforeEach and execution time.

Deliverable: 50–80 lines of code + short reflection on trade-offs.

Reflective Programming in Python

Introspection, dynamic invocation, decorators, proxies, metaclasses and plugin discovery

Python model

Everything is an object: modules, classes, functions, methods, annotations and even classes themselves

```
obj.__class__  
obj.__dict__  
dir(obj)  
getattr(obj, "name")  
inspect.signature(func)  
type("Plugin", (Base,), {...})
```

Why Python is a good reflection laboratory

Python exposes runtime structure directly, with less ceremony than Java.

Everything is inspectable

Classes, modules and functions have runtime attributes such as `__dict__`, `__name__` and `__annotations__`.

Names are dynamic

Attributes can be read, created or replaced at runtime using `getattr` and `setattr`.

Functions are objects

Decorators, callbacks and monkey patches manipulate functions as data.

```
class ReportService:
    version = "1.0"

    def export(self, report):
        return f"report={report}"

service = ReportService()

print(type(service))
print(service.__dict__)
print(ReportService.__dict__.keys())
print(callable(service.export))
```

Teaching message

In Java, reflection usually feels like a special API.

In Python, reflection feels like part of the object model. This makes it powerful for teaching, but also easier to abuse.

Design rule

Use reflection to build flexible boundaries, not to hide poor design.

Introspection: observe objects and classes

The program can ask objects about their runtime structure.

```
class User:
    role = "student"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def is_adult(self):
        return self.age >= 18

u = User("Ana", 21)

print(type(u))           # <class '__main__.User'>
print(u.__class__.__name__) # User
print(u.__dict__)        # instance attributes
print(dir(u))            # available attributes/methods
```

type(obj)

Runtime class of an object.

obj.__dict__

Attributes stored directly in the instance.

dir(obj)

Names visible from the object: fields, methods and inherited members.

callable(x)

Checks whether x can be called as a function/method.

Dynamic access: getattr, setattr, hasattr

Dynamic attribute access is Python's simplest form of reflective programming.

```
class CsvReportService:
    def export(self, path):
        return f"Exporting CSV from {path}"

service = CsvReportService()
method_name = "export"

if hasattr(service, method_name):
    method = getattr(service, method_name)
    result = method("students.csv")

setattr(service, "format", "csv")
print(service.format)
```

Equivalent design idea

Instead of hard-coding `service.export(...)`, the method can be selected from configuration, a command line argument or a plugin registry.

getattr(obj, name)

Retrieve an attribute by name.

setattr(obj, name, value)

Create or replace an attribute dynamically.

hasattr(obj, name)

Check before using dynamic access.

The inspect module: signatures and source

Python can inspect functions, classes and call signatures at runtime.

```
import inspect

def generate_report(path: str, format: str = "pdf") -> bytes:
    '''Generate a report from a dataset.'''
    return b"..."

sig = inspect.signature(generate_report)
print(sig)
# (path: str, format: str = 'pdf') -> bytes

for name, param in sig.parameters.items():
    print(name, param.annotation, param.default)

print(inspect.getdoc(generate_report))
```

Framework use

Validate handler parameters in a web framework or CLI framework.

Testing use

Discover test functions and report better error messages.

AI/tooling use

Build tool schemas from function signatures and annotations.

Warning: source inspection may fail for built-ins, generated functions, optimized code or packaged environments.

Dynamic imports and plugin discovery

A plugin system can load code from names stored in configuration.

```
# config.yaml
# exporter: "plugins.csv_exporter:CsvExporter"

import importlib

def load_class(spec: str):
    module_name, class_name = spec.split(":")
    module = importlib.import_module(module_name)
    return getattr(module, class_name)

ExporterClass = load_class("plugins.csv_exporter:CsvExporter")
exporter = ExporterClass()
exporter.export("students.csv")
```

Pattern connection

Factory + plugin architecture.

Benefit

New implementations without changing core code.

Risk

Loading arbitrary modules is a security boundary.

Safer version: restrict plugin packages, validate interfaces, catch import errors and test failure cases.

Decorators: runtime wrapping as lightweight AOP

A decorator receives a function and returns a new function with extra behavior.

```
import time
from functools import wraps

def timed(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        try:
            return fn(*args, **kwargs)
        finally:
            dt = (time.perf_counter() - t0) * 1000
            print(f"{fn.__name__} took {dt:.2f} ms")
    return wrapper

@timed
def load_students(path):
    return open(path).read().splitlines()
```

AOP vocabulary

Join point

The call to `load_students(...)`.

Advice

The wrapper code before/after the call.

Pointcut

In Python: often explicit `@decorator` usage.

Cross-cutting concern

Timing, logging, caching, authorization.

Proxy/interception with `__getattr__`

Python objects can intercept attribute lookup and delegate dynamically.

```
class LoggingProxy:
    def __init__(self, target):
        self._target = target

    def __getattr__(self, name):
        attr = getattr(self._target, name)

        if not callable(attr):
            return attr

        def logged(*args, **kwargs):
            print(f"calling {name}")
            result = attr(*args, **kwargs)
            print(f"finished {name}")
            return result

        return logged

service = LoggingProxy(RealReportService())
service.export("students.csv")
```

Design connection

This is similar to Java dynamic proxies, but Python does not require a declared interface. Calls are intercepted by attribute lookup.

Good for

Logging, metrics, lazy loading, remote objects.

Danger

Hidden behavior can surprise readers and tools.

Rule

Keep proxies small and explicit.

Classes are objects: creating classes dynamically

In Python, `type` can inspect the class of an object — and also create a class.

```
def export(self, report):
    return f"[{self.format}] {report}"

JsonExporter = type(
    "JsonExporter",      # class name
    (object,),           # base classes
    {
        "format": "json",
        "export": export,
    }
)

plugin = JsonExporter()
print(plugin.export("students"))
print(plugin.__class__.__name__)
```

`type(obj)`

Returns the class of `obj`.

`type(name, bases, dict)`

Creates a new class dynamically.

Use cases

ORM models, serializers, plugin systems, test doubles.

Exam question: when is dynamic class creation better than a normal class definition?

Metaclasses: customizing class creation

A metaclass controls what happens when a class object is created.

```
class PluginMeta(type):
    registry = {}

    def __new__(mcls, name, bases, namespace):
        cls = super().__new__(mcls, name, bases, namespace)
        if name != "Plugin":
            PluginMeta.registry[name] = cls
        return cls

class Plugin(metaclass=PluginMeta):
    pass

class CsvExporter(Plugin):
    pass

class JsonExporter(Plugin):
    pass

print(PluginMeta.registry.keys())
```

What it does

Intercepts creation of classes, not instances.

Why it matters

Automatic registration, validation, ORM model construction.

Use carefully

Metaclasses are powerful but difficult for beginners to debug.

Simpler alternatives often exist: decorators, explicit registries, base-class `__init_subclass__`.

Descriptors and properties: intercept attribute access

Python lets objects define how attribute read/write/delete operations behave.

```
class Positive:
    def __set_name__(self, owner, name):
        self.name = "_" + name

    def __get__(self, obj, owner):
        return getattr(obj, self.name)

    def __set__(self, obj, value):
        if value <= 0:
            raise ValueError("must be positive")
        setattr(obj, self.name, value)

class Product:
    price = Positive()

    def __init__(self, price):
        self.price = price
```

Descriptor

Object implementing `__get__`, `__set__` or `__delete__`.

@property

Common descriptor-based interface for computed fields.

Framework use

Validation, ORMs, dataclasses, computed attributes.

Descriptors are a form of controlled intercession: normal-looking attribute access triggers custom behavior.

Monkey patching: runtime modification

Python can replace behavior at runtime — useful in tests, risky in production.

```
class PaymentGateway:
    def charge(self, amount):
        return "real payment"

def fake_charge(self, amount):
    return "fake payment"

# Replace method dynamically
PaymentGateway.charge = fake_charge

gateway = PaymentGateway()
print(gateway.charge(100)) # fake payment
```

Prefer explicit dependency injection or test fixtures when possible.

When it helps

Testing

Replace slow or external dependencies.

Compatibility patches

Patch third-party libraries temporarily.

Why it is dangerous

Hidden global change

All users of the class see the modification. Debugging order-dependent tests becomes hard.

Python reflection vs Java reflection

Same underlying idea, different language culture and trade-offs.

Topic	Java	Python
Type model	Static types + runtime metadata	Dynamic objects + runtime attributes
Invocation	Method.invoke(...)	getattr(obj, name)(...)
Annotations	@Test, @Entity, @Service	__annotations__, decorators
Interception	Dynamic proxies / bytecode tools	decorators, __getattr__, descriptors
Risk	Reflection exceptions, modules, access checks	hidden mutation, monkey patching, tooling surprises

Course takeaway

The design question is not “which API is more powerful?”, but “where should dynamic behavior be allowed?”

Mini-lab: build a Python plugin runner

Students implement a small reflection-based plugin system.

```
# plugins/uppercase.py
class UppercasePlugin:
    name = "upper"
    def run(self, text):
        return text.upper()

# runner.py
plugin = load_class("plugins.uppercase:UppercasePlugin")()
print(plugin.run("hello"))
```

Reflection concepts practiced: `importlib`, `getattr`, `inspect.signature`, decorators, dynamic invocation and error handling.

Optional extension: discover all plugins in a package and register them by their name attribute.

Required tasks

1

Load a plugin class from a string.

2

Verify it has `run(self, text)`.

3

Catch import/signature errors.

4

Add a `@timed` decorator.

Takeaways

What students should remember after the lecture.

Reflection

turns program elements into runtime data.

Python

exposes reflection directly through its object model.

Proxies/AOP

intercept calls to modularize cross-cutting concerns.

Java

uses explicit metadata objects: Class, Method, Field and Annotation.

Annotations/decorators

let programmers declare behavior consumed by tools or frameworks.

Design rule

centralize reflection; keep the application core strongly typed and testable.

Suggested references and further reading

Use these for optional reading or assignment prompts.

Gregor Kiczales et al., The Art of the Metaobject Protocol, MIT Press, 1991.

Brian Cantwell Smith, Reflection and Semantics in Lisp, 1982.

Java Platform documentation: `java.lang.reflect` and `java.lang.annotation`.

Spring Framework reference: Core technologies, AOP and dependency injection.

Byte Buddy documentation: runtime code generation for Java.

Python documentation: `inspect`, `importlib`, `dataclasses`, `descriptors` and the data model.

PEP 318: Decorators for Functions and Methods.

PEP 487: Simpler customization of class creation.

Attribution

This deck extends the uploaded reflective-programming slides by Pedro García López, URV, using the same white-background teaching style.